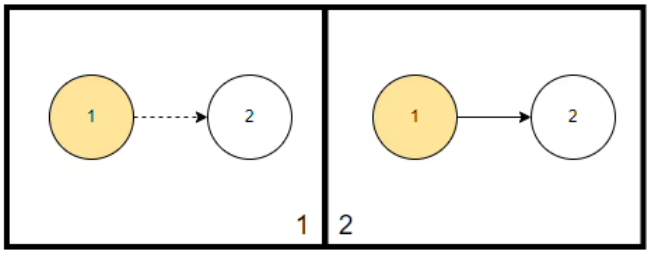
24339번 - 가희와 쓰레기 놀이

시간 제한	메모리 제한	제출	정답	맞힌 사람	정답 비율
3.5 초	512 MB	259	68	39	23.926%

문제

가희는 친구들과 함께 쓰레기 수집 놀이를 하고 있습니다. 이 놀이를 이해하는 데 필요한 용어를 설명하겠습니다.



[그림 1-1(좌), 그림 1-2 (우)] object 1에서 object 2로 연결하는 연결 관계

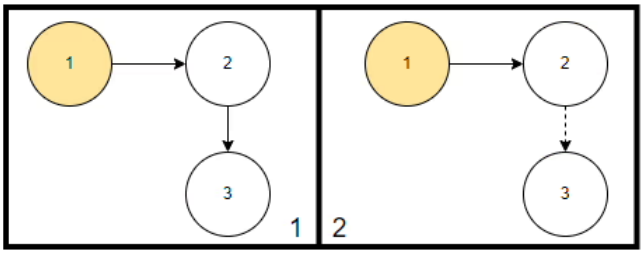
먼저 연결 관계란 object에서 object를 연결하는 관계를 의미합니다. 예를 들어, [그림 1-1], [그림 1-2]는 object 1에서 object 2로 연결하는 관계를 나타냅니다. 연결 관계는 약한 연결 관계와, 강한 연결 관계가 있습니다. **이 중, object 1에서 object 2로 연결하는 약한 연결 관계는 [그림 1-1]에, 1에서 2로 연결하는 강한 연결 관계는 [그림 1-2]에 표시되어 있습니다.**

가희와 친구들이 수행할 수 있는 연산에 대한 설명은 아래와 같습니다.

- M
  - **root로부터 강한 연결 관계를 통해서만 도달할 수 있는** object만 남기고 나머지 object를 모두 제거합니다.
- m
  - **root로부터 강한 연결, 혹은 약한 연결 관계를 통해서 도달할 수 있는** object만 남기고 나머지 object를 모두 제거합니다.
- MADE `object_id is_root`
  - id가 `object_id` 인 object를 추가합니다. `is_root` 의 값이 `ROOT` 이면 해당 object는 root임을 의미합니다.
- ADD `ref_id object_id_1 -> object_id_2`
  - `object_id` 가 `object_id_1` 인 객체에서 `object_id` 가 `object_id_2` 인 객체를 연결하는, 연결 관계 id가 `ref_id` 인 약한 연결 관계를 추가합니다.
- ADD `ref_id object_id_1 => object_id_2`
  - `object_id` 가 `object_id_1` 인 객체에서 `object_id` 가 `object_id_2` 인 객체를 연결하는, 연결 관계 id가 `ref_id` 인 강한 연결 관계를 추가합니다.
- REMOVE `ref_id`
  - 연결 관계 id가 `ref_id` 인 연결 관계를 제거합니다.

M과 m 연산이 일어날 때마다, 놀이에 있는 모든 object에 대해, root로부터 도달이 가능한지를 판단하게 됩니다. root에서 object x가 도달 가능하다는 것은, 아래 조건 중 하나 이상을 만족합니다.

- object x는 root입니다.
- root에서 조건을 만족하는 연결 관계를 거쳐서 object x에 도달할 수 있습니다.



[그림 2-1 (좌), 그림 2-2 (우)]

[그림 2-1], [그림 2-2]에서 root는 노란색으로 칠해져 있습니다. [그림 2-1]은 1에서 2로 연결하는 강한 연결 관계와 2에서 3으로 연결하는 강한 연결 관계가 있음을 나타냅니다. root로부터 도달 가능한 object는 1, 2, 3입니다. [그림 2-2]는 1에서 2로 연결하는 강한 연결 관계와 2에서 3으로 연결하는 약한 연결 관계가 있음을 나타냅니다. 강한 연결 관계나 약한 연결 관계를 통해서 root로부터 도달 가능한 object는 1, 2, 3입니다. **하지만, 강한 연결 관계를 통해서만 도달할 수 있는 object는 1과 2뿐입니다.** root인 1에서 root가 아닌 3으로 가기 위해서는 2에서 3으로 연결하는 약한 연결 관계를 반드시 통해야 하기 때문입니다.

이 놀이의 규칙은 다음과 같습니다.

- 게임이 시작되기 전에는 생성된 object 중에 root 역할을 하는 object가 있을 수도 없을 수도 있습니다.
- 게임이 시작되기 전에는 object와 object를 연결하는 **어떠한 연결 관계도 존재하지 않습니다.**
- 가희와 친구들은 게임이 진행되는 동안 여러 연산을 할 수 있습니다.

object가 제거될 때, 제거된 object로부터 다른 object를 연결하는 연결 관계와 다른 object에서 제거된 object, 그리고 제거된 object로부터 제거된 object로 연결되는 **연결 관계들이 모두 제거**됩니다. 게임을 시작하기 전 생성된 object들과 게임 중에 가희와 친구들이 한 연산들이 시간 순서대로 주어집니다. 가희와 친구들이 연산을 수행할 때마다, 연산을 수행하고 남은 object의 수를 출력해 주세요.

입력

첫 번째 줄에 게임이 시작되기 전 생성된 object의 개수 O와 게임에서 가희와 친구들이 수행한 연산의 개수 E가 공백으로 구분되어 주어집니다.

다음 O개의 줄에는 `object_id` 와 `is_root` 가 공백으로 구분되어 주어집니다.

이때 `is_root` 는 해당 object가 root라면 ROOT, 아니라면 -로 주어지며, **주어지는 `object_id` 는 모두 다릅니다.**

다음 E개의 줄에는 문제에서 설명한 연산 중 하나가 **발생한 시간 순서대로** 주어집니다.

출력

M과 m 연산이 주어질 때마다 해당 연산을 수행하고 난 후에 남아 있는 object의 개수를 한 줄에 하나씩 출력해 주세요.

제한

- $1 \le O \le 2 \times 10^5$
- $1 \le E \le 2 \times 10^5$
- 객체 id가 `object_id` 인 객체가 생성된 상태에서 id가 `object_id` 인 객체를 생성하는 연산은 주어지지 않습니다.
- 연결 번호 `ref_id` 인 연결 관계가 생성된 상태에서 연결 번호가 `ref_id` 인 연결 관계를 생성하는 연산은 주어지지 않습니다.
- 연결 번호 `ref_id` 인 연결 관계가 없는 상태에서 연결 번호가 `ref_id` 인 연결 관계를 삭제하는 연산은 주어지지 않습니다.
- `object_id_1` 과 `object_id_2` 를 연결하는 연결 관계를 추가할 때 객체 고유 번호가 `object_id_1` 인 object와 고유 번호가 `object_id_2` 인 object는 항상 존재합니다.
- 1 ≤ 연산 M과 m을 하는 횟수의 총합 ≤ 20
- `object_id` 와 `ref_id` 는 1 이상 10^9 이하 정수입니다.
- 놀이가 시작되기 전 생성 되어 있는 O개의 객체 고유 번호는 **중복되지 않습니다.**

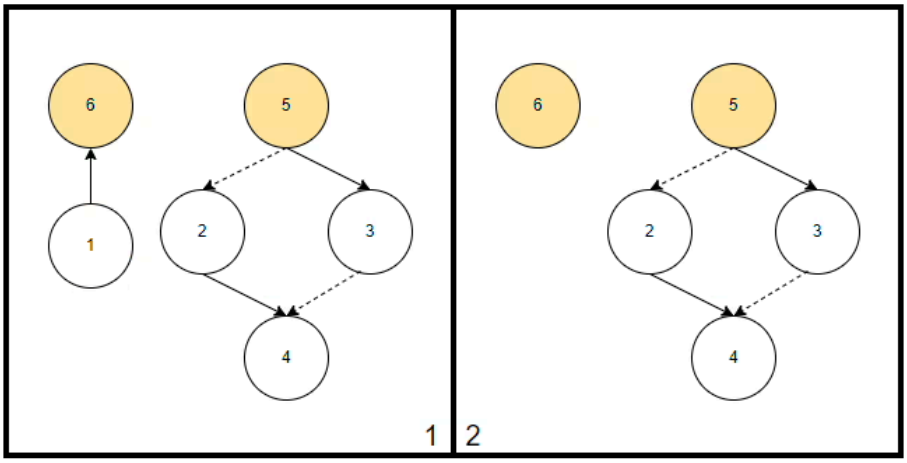
예제 입력 1 복사

```
2 11
5 ROOT
1 -
MADE 6 ROOT
MADE 2 -
ADD 1 5 -> 2
MADE 3 -
ADD 2 5 => 3
MADE 4 -
ADD 4 2 => 4
ADD 3 3 -> 4
ADD 10 1 => 6
m
M
```

예제 출력 1 복사

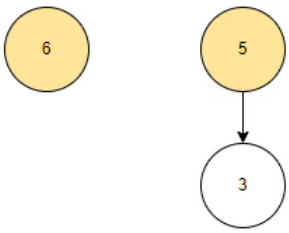
```
5
3
```

ADD 10 1 => 6 연산을 수행한 후에 상태는 그림 3-1과 같습니다.



[그림 3-1(좌), 그림 3-2(우)] 예제 1의 연산에 따른 게임의 상태

m 연산을 수행한 후에 *object_id*가 1인 object는 제거됩니다. 이는 노랗게 칠한 root로부터 1까지 도달하는 방법이 없기 때문입니다. 그림 3-2는 m 연산이 수행된 후의 상태를 나타냅니다. 다음에 M 연산이 수행됩니다. **root로부터 2번과 4번은 점선으로 표시된 약한 연결 관계를 통하지 않고 가는 방법이 없습니다.** 따라서, M 연산이 수행된 후에 *object_id*가 2인 object와 4인 object는 제거됩니다.



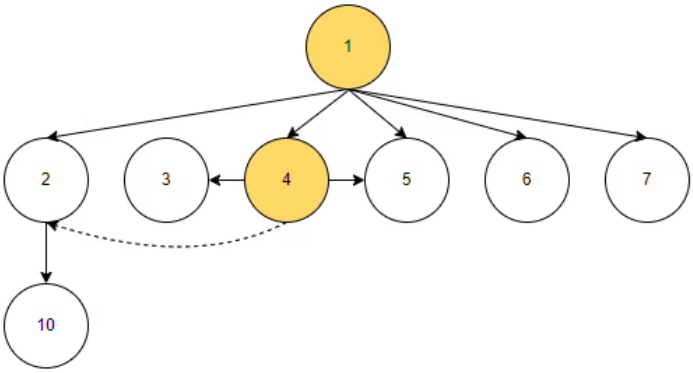
[그림 4] 예제 1에서 M 연산이 발생한 후 상태

예제 입력 2 복사

```
7 20
1 ROOT
3 -
2 -
4 ROOT
5 -
6 -
7 -
ADD 1 1 => 2
ADD 2 1 => 3
ADD 3 1 => 4
ADD 4 1 => 5
ADD 5 1 => 6
ADD 6 1 => 7
ADD 7 4 -> 2
ADD 8 4 => 3
ADD 9 4 => 5
MADE 10 -
ADD 10 2 => 10
REMOVE 2
M
MADE 11 -
ADD 12 10 => 11
ADD 11 4 => 11
REMOVE 1
M
REMOVE 11
m
```

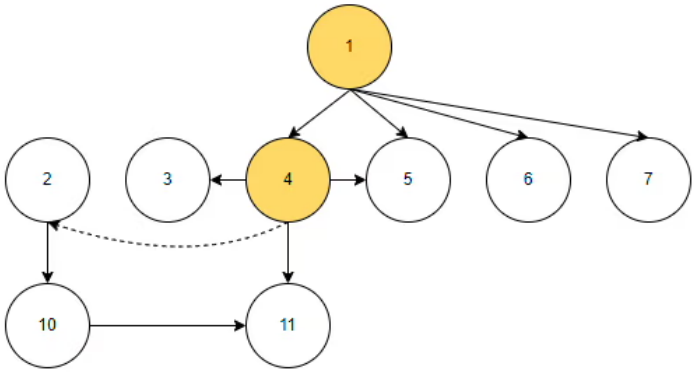
예제 출력 2 복사

```
8
7
6
```



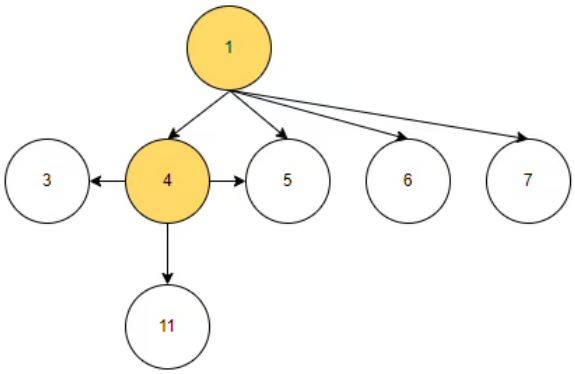
[그림 5] 첫 M 연산이 수행되기 직전 상태

생성된 모든 object는 노란색으로 칠한 root로부터 강한 연결 관계만을 통해서 도달할 방법이 있으므로, 모든 object는 제거되지 않습니다.



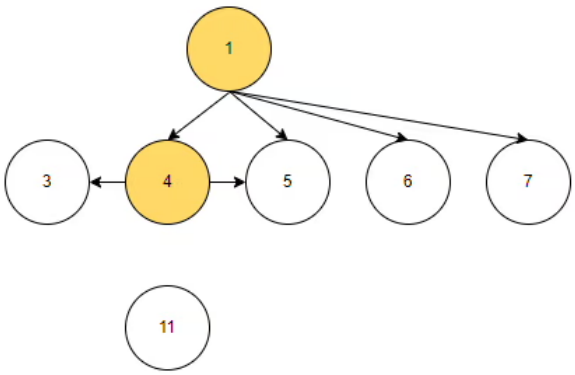
[그림 6] 두 번째 M 연산이 수행되기 직전의 상태

그림 6을 보면 2, 10은 약한 연결 관계 (점선으로 표시된 화살표)를 거치지 않고 가는 방법이 없습니다. 따라서 2와 10이 제거되게 됩니다. 2와 10이 제거된 후에는 게임의 상태가 아래와 같이 변합니다.



[그림 7] 두 번째 M 연산이 수행된 후 게임의 상태

다음에 4와 11을 잇는 *ref_id*가 11인 연결 관계가 제거되면 게임의 상태는 아래와 같이 바뀌게 됩니다.



[그림 8] REMOVE 11 연산 후 게임의 상태

이후에 m이 수행됩니다. 11은 root인 4와 1로부터 약한 연결을 통해서도 갈 수 없으므로, 11이 제거되게 됩니다.

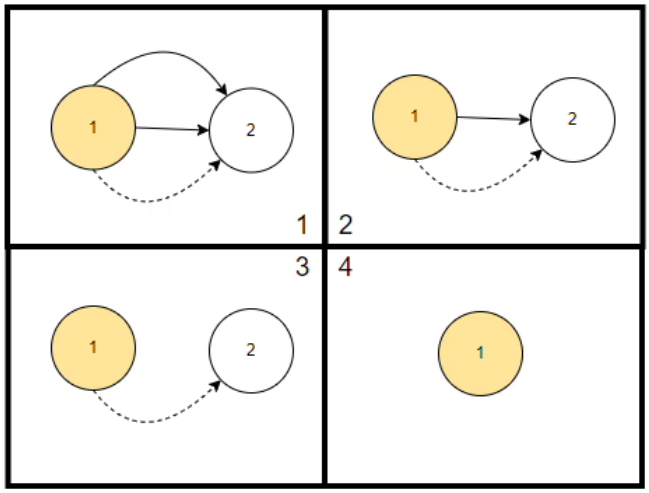
예제 입력 3 복사

```
2 8
1 ROOT
2 -
ADD 1 1 => 2
ADD 2 1 => 2
ADD 3 1 -> 2
M
REMOVE 1
M
REMOVE 2
M
```

예제 출력 3 복사

```
2
2
1
```

그림 9의 1, 2, 3, 4는 각각 1번째 M이 호출되기 직전, 2번째 M이 호출되기 직전, 3번째 M이 호출되기 직전, 3번째 M이 호출된 후를 나타냅니다.



[그림 9-1(왼쪽 위), 그림 9-2 (오른쪽 위), 그림 9-3(오른쪽 아래), 그림 9-4(왼쪽 아래)] 예제 3에 대한 설명

1번째와 2번째 M이 호출될 때 2가 제거되지 않은 이유는 root인 1로부터 2까지 가는 방법 중, 강한 연결 관계를 통해서 가는 방법이 있었기 때문입니다. 반면에 3번째 M이 호출되기 직전에는 **root로부터 강한 연결 관계를 통해서 2로 갈 방법이 존재하지 않습니다.** 따라서 3번째 M이 호출되고 난 후에 *object_id*가 1인 object가 지워지게 됩니다.

예제 입력 4 복사

```
3 15
105713756 ROOT
512671000 -
116239500 -
MADE 7563 ROOT
ADD 1063 105713756 => 512671000
ADD 77251637 512671000 => 7563
MADE 6 -
ADD 71 7563 => 6
ADD 30 6 => 116239500
ADD 10 512671000 -> 116239500
M
REMOVE 30
m
ADD 30 6 -> 116239500
M
REMOVE 1063
ADD 1064 105713756 -> 512671000
M
```

예제 출력 4 복사

```
5
5
4
3
```

노트

root가 없는 경우가 있거나, 자신이 자신을 가리키는 self-loop가 있는 경우도 있음을 조심하세요.

출처

Contest (/category/45) > BOJ User Contest (/category/984) > 가희와 함께 하는 코딩 테스트 (/category/529) > 가희와 함께 하는 3회 코딩테스트 (/category/detail/3014) 3번

- 문제를 만든 사람: chogahui05 (/user/chogahui05)
- 데이터를 추가한 사람: wapas (/user/wapas)
- 문제를 검수한 사람: 9507ym (/user/9507ym), amsminn (/user/amsminn), bnb2011 (/user/bnb2011), jthis (/user/jthis), jyheo98 (/user/jyheo98), pjshwa (/user/pjshwa), swoon (/user/swoon), tony9402 (/user/tony9402)